



Few-Shot Speech Deepfake Detection Adaptation with Gaussian Processes

Neta Glazer, David Chernin, Idan Achituve, Sharon Gannot, Ethan Fetaya

¹Bar Ilan University, Israel

netaglazer@gmail.com, ethan.fetaya@biu.ac.il

Abstract

Recent advancements in Text-to-Speech (TTS) models, particularly in voice cloning, have intensified the demand for adaptable and efficient deepfake detection methods. As TTS systems continue to evolve, detection models must be able to efficiently adapt to previously unseen generation models with minimal data. This paper introduces ADD-GP, a few-shot adaptive framework based on a Gaussian Process (GP) classifier for Audio Deepfake Detection (ADD). We show how the combination of a powerful deep embedding model with the Gaussian processes flexibility can achieve strong performance and adaptability. Additionally, we show this approach can also be used for personalized detection, with greater robustness to new TTS models and one-shot adaptability. To support our evaluation, a benchmark dataset is constructed for this task using new state-of-the-art voice cloning models.

Index Terms: Audio Deepfake Detection, Few-Shot Adaptation, Gaussian Processes

1. Introduction

The rapid advancements in text-to-speech (TTS) generation have significantly improved the ability to synthesize highly realistic speech from textual input [1, 2]. Recent models also excel in voice cloning, enabling the generation of speech that closely mimics a specific speaker’s voice with just a single reference utterance [3, 4]. This breakthrough has significantly enhanced applications in accessibility, entertainment, and human-computer interaction.

However, these advancements also introduce significant security and ethical concerns, particularly in the realm of voice cloning. With only a short audio sample, modern voice cloning models can generate speech that is nearly indistinguishable from the original speaker’s voice. This capability has been exploited in various malicious activities, including fraud, misinformation, and identity theft. For instance, fraudsters have successfully mimicked corporate executives to authorize fraudulent financial transactions, leading to substantial financial losses. Similarly, scammers have used synthetic voices to pose as family members in distress, coercing victims into transferring money under false pretenses [5].

This problem is addressed by Audio Deepfake Detection (ADD) algorithms. We observed that current ADD detectors easily detect deepfakes from known TTS models. However, they may generalize poorly to deepfakes from unseen generation models. This is a significant security concern as new models are introduced at an accelerating rate. Moreover, advancement in parameter efficient fine-tuning, e.g., LoRA [6], allows attackers to modify an existing TTS model on a relatively small amount of data. To address this challenge, a detection system

must be capable of efficiently adapting to novel deepfake generation methods with minimal data, allowing it to continuously evolve against new threats. We also note that many leading commercial TTS models, such as 11Labs, are only accessible via paid APIs. This limits the feasibility of large-scale data collection for retraining, further emphasizing the need for efficient few-shot adaptation.

In this paper, we introduce a new system for adaptive deepfake detection based on Gaussian Processes (GP), which we name ADD-GP. We use a Deep Kernel Learning (DKL) framework, based on XLS-R features [7], to improve the generalization capability of detection models. Gaussian Processes (GPs) are well-suited for few-shot detection due to their non-parametric nature and ability to provide uncertainty estimates. By integrating GPs with deep neural networks through Deep Kernel Learning (DKL), we enable the model to learn highly expressive feature representations while maintaining the flexibility and robustness of GPs. Finally, we explore the use of our framework for personalized deepfake detection. This shows improved robustness to new TTS models and better adaptability even in cases where only a single generated example is available.

Our key contributions include:

- A novel method for audio deepfake detection, enabling effective few-shot adaptation for new TTSs.
- A state-of-the-art personalized few-shot deepfake detection with greater robustness and adaptability.
- The introduction of **LibriFake**, a dataset specifically designed for evaluating few-shot and one-shot deepfake audio detection approaches on novel TTS models. To support future research and ensure the reproducibility of our results, we have made both our source code and datasets publicly available at: <https://github.com/NetaGlazer/ADD-GP>

2. Background

We provide a brief introduction to Gaussian Process (GP) classifiers, which form the foundation of our proposed method. Scalars are denoted with lowercase letters (e.g., x), vectors with boldface lowercase letters (e.g., $\mathbf{x}$), and matrices with boldface uppercase letters (e.g., $\mathbf{X}$). In general, $\mathbf{y} = [y_1, \dots, y_N]^T$ represents the vector of labels, and $\mathbf{X} \in \mathbb{R}^{N \times d}$ represents the design matrix comprising N data points of dimension d , where the i -th row is $\mathbf{x}_i$.

2.1. Gaussian Processes

Gaussian Processes (GPs) define a distribution over functions. A GP model is fully specified by a mean function $m(\mathbf{x})$ and

a covariance function (kernel) $k(\mathbf{x}, \mathbf{x}')$. We say that $\mathbf{f} \sim \mathcal{GP}(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}'))$ if for a given dataset $\mathbf{X}$, the function values $\mathbf{f} = [f(\mathbf{x}_1), \dots, f(\mathbf{x}_N)]^T$ are normally distributed, i.e., $\mathbf{f} \sim \mathcal{N}(\boldsymbol{\mu}, \mathbf{K})$, where $\mu_i = m(\mathbf{x}_i)$ and $\mathbf{K}_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ [8]. Commonly, the mean function is the constant zero function. The predictive power of GPs mainly comes from the kernel which encodes similarity between the data points. For regression tasks if the likelihood $p(\mathbf{y}|\mathbf{f}, \mathbf{X})$ takes a Gaussian form, the posterior distribution $p(\mathbf{f}|\mathbf{y}, \mathbf{X})$ is a Gaussian with a closed-form formula for the mean and variance. At inference, given a new data point $\mathbf{x}_*$, the predictive distribution $p(\mathbf{f}_*|\mathbf{x}_*, \mathbf{y}, \mathbf{X}) = \int p(\mathbf{f}_*|\mathbf{x}_*, \mathbf{f})p(\mathbf{f}|\mathbf{y}, \mathbf{X})d\mathbf{f}$ is also a Gaussian with parameters given by:

$$p(\mathbf{f}_*|\mathbf{x}_*, \mathbf{y}, \mathbf{X}) = \mathcal{N}(\mu_*, \sigma_*^2) \quad (1)$$

$$\mu_* = \mathbf{k}_*^T (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{y} \quad (2)$$

$$\sigma_*^2 = k_{**} - \mathbf{k}_*^T (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{k}_* \quad (3)$$

where $k_{**} = k(\mathbf{x}_*, \mathbf{x}_*)$, and $\mathbf{k}_*[i] = k(\mathbf{x}_i, \mathbf{x}_*)$. This means that given a kernel and a dataset, the GP prediction is given by a closed-form solution without any optimization.

2.2. Gaussian Process Classification (GPC)

Unlike GP regression with a Gaussian likelihood, in Gaussian Process Classification (GPC), the likelihood $p(\mathbf{y}|\mathbf{f}, \mathbf{X})$ is non-Gaussian, making exact posterior inference intractable. To address this, approximate inference techniques, such as variational inference (VI) [9, 10, 11], or Markov chain Monte Carlo (MCMC) [12, 13, 14, 15], are commonly used. Here, we follow the approach presented in [16], although our approach can be combined with any standard GP classifier. It is important to note that this approach still uses a closed-form solution for the posterior and at inference.

2.3. Deep Kernel Learning (DKL)

The performance of GP models is heavily dependent on the choice of kernel function $k(\mathbf{x}_i, \mathbf{x}_j)$ with standard kernels often failing to capture meaningful semantic similarities in complex data modalities such as images and speech. Deep Kernel Learning (DKL) addresses this limitation by combining classical kernels with a neural network transformation [17, 18]. For example, it is common to use the RBF kernel on features computed by a deep network g_θ :

$$k_\theta(\mathbf{x}_i, \mathbf{x}_j) = \sigma \cdot \exp \left(-\frac{\|g_\theta(\mathbf{x}_i) - g_\theta(\mathbf{x}_j)\|^2}{2\ell^2} \right). \quad (4)$$

Here, the kernel parameters are (1) the neural network parameters θ , (2) the length scale ℓ and (3) the output scale σ . Typically, all the parameters are learned using a Bayesian loss, such as the log marginal likelihood or the evidence lower bound (ELBO).

2.4. Audio Deepfake Detection

Audio Deepfake Detection (ADD) methods generally follow two main architectures: pipeline-based and end-to-end (E2E) based approach. In pipeline-based models [19], the system is split into two distinct components: a front-end feature extractor and a back-end classifier [20]. The front-end extracts features such as Mel-Frequency Cepstral Coefficients (MFCC), Linear Frequency Cepstral Coefficients (LFCC), or embeddings from

a deep neural network, e.g., wav2vec2, which are then passed to an independent back-end classifier for classification.

Some E2E models, like SSL-AASIST [21] still use pre-trained feature extractors, but instead of keeping them fixed, they fine-tune both the front-end feature extractor and the back-end classifier together. Other E2E models, such as RawGAT-ST [22] and RawNet3 [23], learn directly from raw waveforms, extracting features and classifying them within the same model without a separate feature extraction step.

Wav2Vec2 [24], HuBERT [25], and XLS-R [7] have become standard backbone models in modern ADD systems due to their strong ability to extract meaningful audio representations [26]. These speech foundation models are widely used in both pipeline-based and end-to-end architectures, either as fixed feature extractors or fine-tuned alongside classifiers. Compared to handcrafted features and traditional deepfake detection models, self-supervised speech models offer stronger generalizability, especially when tested on unseen datasets [27].

3. Methods

We introduce our framework that uses a Dirichlet-based GP classifier [16] as a back-end, on features extracted from XLS-R [7], a Wav2Vec2 based self-supervised model, as the front-end.

3.1. Datasets Generation

To evaluate our method, we introduce LibriFake, a large-scale deepfake benchmark derived from LibriSpeech [28]. For each sample in LibriSpeech, we generate a corresponding synthetic version using voice cloning with the original text transcription, effectively replicating the dataset for each TTS model. The dataset is then split into training and testing subsets, with $\sim 15\%$ of speakers in the test set, ensuring that no samples from the same speaker appear in both sets. This separation is designed to test the model's ability to generalize to new speakers. We used: (i) yourTTS [3], (ii) Whisper-Speech [29], (iii) Vall-e-x [30], (iv) F5-TTS [4], and (v) 11Labs.¹ When evaluating generalization to unseen TTS, we found that detection performance was significantly worse on 11Labs. Therefore, we selected it as the unseen TTS for our few-shot adaptation experiments. For a personalized out-of-distribution evaluation, we also selected 30 speakers from the VoxCeleb dataset [31] and generated fake samples from them using the same process we employed to create LibriFake. Because VoxCeleb, unlike LibriSpeech, does not include transcripts, we first transcribed the speakers' audio with Whisper-Large-v2. These transcriptions were then used for the TTS text conditioning.

3.2. GP Deepfake Detection

In the first phase, we train a Gaussian Process (GP) classifier to distinguish between real and fake speech samples from our dataset. We used the training set of all available TTS models, except for the 11Labs model, which was left unseen during training, and 1000 random training examples excluded from the kernel learning phase. We used the RBF kernel on features computed by XLS-R, which was frozen except for the last block. This means that during training, we only update the last XLS-R block, the lengths scale, and the output scale. At each iteration, we draw 80 samples, embed them via XLS-R, build a GP classifier using the RBF-kernel on these features (eq. 4), compute the log-marginal likelihood, backpropagate, and update the ker-

¹<https://elevenlabs.io/>

Table 1: Evaluation of 11Labs using Wav2Vec2-2B. Equal Error Rate (EER) in percentage averaged over three random seeds.

	ID TTS	OOD 11Labs	5-Shots	10-Shots	20-Shots	50-Shots	100-Shots	100-Shots ID	Params
SSL-AASIST-FT	0.2%	42.48%	16.17 ± 0.62	15.55 ± 0.01	12.56 ± 0.27	11.73 ± 0.75	10.97 ± 0.40	2.3%	2b
SSL-AASIST-FT MixPro	0.2%	42.48%	17.44 ± 0.09	14.10 ± 0.61	12.22 ± 0.27	8.26 ± 0.09	5.76 ± 0.27	5.42%	2b
SSL-AASIST-Lora-FT	0.2%	42.48%	22.98 ± 1.33	20.27 ± 0.09	20.62 ± 0.39	20.35 ± 0.01	20.20 ± 0.01	1.02%	5m
SSL-AASIST-Lora-FT MixPro	0.2%	42.48%	24.59 ± 1.5	21.25 ± 0.03	21.04 ± 0.35	19.79 ± 0.02	19.59 ± 0.02	1.33%	5m
OWM	0.2%	42.48%	26.02 ± 1.67	23.97 ± 0.04	20.37 ± 0.56	11.54 ± 0.09	7.85 ± 0.16	7.29%	2b
OWM MixPro	0.2%	42.48%	19.00 ± 1.57	13.02 ± 0.08	10.12 ± 0.34	7.85 ± 0.07	3.93 ± 0.15	1.42%	2b
RWM	0.2%	42.48%	20.04 ± 1.15	16.94 ± 0.29	15.98 ± 0.39	5.09 ± 0.74	4.41 ± 0.09	0.51%	2b
RWM MixPro	0.2%	42.48%	14.75 ± 0.7	12.30 ± 0.5	6.97 ± 0.18	4.92 ± 0.68	1.24 ± 0.05	0.71%	2b
CD-ADD	0.4%	35.12%	32.37 ± 1.63	28.28 ± 0.67	15.29 ± 0.39	9.98 ± 1.31	7.92 ± 0.12	23.68%	2b
CD-ADD MixPro	0.4%	35.12%	28.93 ± 1.51	26.45 ± 0.57	19.21 ± 0.21	8.88 ± 0.96	7.23 ± 0.09	23.68%	2b
ADD-GP (ours)	0.1%	40.31%	13.44 ± 0.01	12.80 ± 0.03	8.72 ± 0.03	4.70 ± 0.04	2.78 ± 0.03	0.1%	—
ADD-GP MixPro (ours)	0.1%	40.31%	7.89 ± 0.01	4.86 ± 0.01	3.31 ± 0.08	1.14 ± 0.01	0.54 ± 0.03	0.1%	—

nel parameters. During the evaluation phase, we fixed our tuned parameters and built a GP classifier using our learned kernel on the 1000 training samples we kept aside. We show pseudocode in Alg. 1.

3.3. Few-Shot Adaptation

As the kernel only measures similarity, it can generalize well. As such, we can keep the kernel fixed and only update the GP with the fixed kernel. As GPs are a non-parametric method, adapting them without any training is straightforward. We simply add the examples from the new TTS to the held-out 1000 training examples and build a new GP classifier with these additional examples, i.e. recomputing lines 10-11 in Alg. 1 with an augmented $\mathbf{z}_{\text{eval}}$.

3.4. MixPro for Few-Shot Adaptation

To further enhance generalization in few-shot scenarios, we incorporate MixPro [32], a data augmentation technique based on MixUp [33], specifically designed for few-shot domain adaptation. MixPro generates additional training examples by blending source and target domain samples in the embedding space.

More precisely, given a neural network embedding function f and two samples $\mathbf{x}_s$ and $\mathbf{x}_t$ from the same class—one from the source domain and one from the target domain—we create mixed examples using the following interpolation:

$$f_{\text{mix}} = (1 - \lambda)f(\mathbf{x}_s) + \lambda f(\mathbf{x}_t), \quad (5)$$

where λ is a random interpolation parameter. In our case, only the fake samples are OOD, so we only mix deepfake samples from the new TTS with deepfake samples from the training TTSs. In our GP few-shot adaptation experiments, we created an augmented dataset that was 20 times larger than the original and combined it with the original samples. For the baseline methods, we iteratively sampled augmented examples at each fine-tuning step.

4. Experiments

4.1. Baselines

We evaluated our approach against SSL-AASIST [34], a widely used baseline known for its strong performance on publicly

available datasets. SSL-AASIST employs the AASIST [21] architecture as the back-end while using XLS-R as the front-end, making it a relevant benchmark for comparison with our XLS-R-based framework.

To adapt SSL-AASIST to few-shot learning, we fine-tuned the model both with and without LoRA, as well as using Orthogonal Weight Matching (OWM) [35], a continual learning method designed to mitigate catastrophic forgetting. Additionally, we incorporated Remember Weight Matching (RWM) [36], a recently introduced continual learning approach that has demonstrated strong few-shot adaptation capabilities. Notably, RWM was originally implemented with an XLS-R backbone, aligning closely with our setup. Furthermore, we implemented the CD-ADD method [26], which has also been shown to support few-shot adaptation. To ensure a fair comparison, all baselines were trained and evaluated using the same XLS-R parameters.

4.2. Few-Shot Adaptation for New TTS Models

We first trained all baselines on our LibriFake training datasets, including all TTS models except for 11Labs, which will be used as the out-of-distribution TTS. We then adapted each baseline with 5, 10, 20, 50, and 100 samples from 11Labs. We also train each method on 11Labs samples using the MixPro method, explained in Sec. 3.4. We run each adaptation method with three different random seeds and present the average results.

Table 1 presents the equal error rate (EER) performance across different settings. First, in the In-Distribution (ID) TTS column, we report the EER on test data from TTS systems seen during training. As expected, all methods perform exceptionally well in this scenario. Next, in the Out-Of-Distribution (OOD) 11Labs setting, we evaluate how well the original models detect deepfake audio from the unseen 11Labs TTS on test speakers. The results indicate that all methods struggle with generalization, leading to poor performance on this task. We then examine k -shot adaptation, where our ADD-GP method demonstrates a significant advantage over all baselines. Additionally, we observe improvement across most methods when incorporating MixPro augmentation. Finally, we test how the model adapted on 100-shot 11Labs samples performs on the original TTS seen during training. All methods, except for our

Table 2: *Personalized evaluation on LibriSpeech (LS), and VoxCeleb (VC) subsets.*

	OOD (LS)	1-shot (LS)	5-shot (LS)	OOD (VC)	1-shot (VC)	5-shot (VC)
SSL-ASSIST-FT	5.92%	5.18%	4.13%	11.07%	10.4%	6.41%
SSL-ASSIST-LORA-FT	19.89%	18.33%	15.81%	48.01%	46.42%	46.42%
CD-ADD	11.24%	6.65%	3.69%	10.03%	9.76%	9.17%
OWM	13.25%	9.04%	7.23%	10.86%	9.11%	6.34%
RWM	7.75%	7.1%	6.06%	12.8%	7.39%	6.69%
ADD-GP (ours)	2.59%	0.61%	0.39%	6.95%	2.03%	0.52%

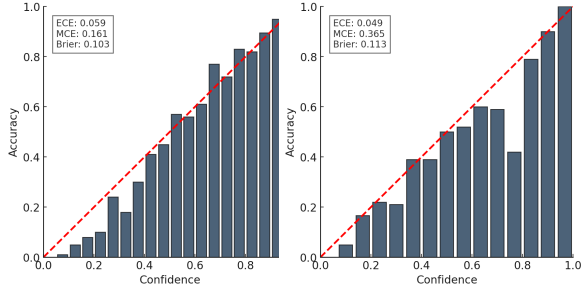


Figure 1: *Calibration curves for different models. On the left is the calibration on the 10-shot experiment. On the right, calibration for personalized 5-shot VoxCeleb experiment.*

ADD-GP, show a decrease in performance on the original TTSs, even when using methods designed to mitigate “catastrophic forgetting”. We note that this aligns with previous works that showed good continual learning results with GP models [14].

4.3. Personalized Deepfake Detection

We also explored personalized deepfake detection to highlight the additional benefits of the GP approach’s flexibility. In this experiment, each method was tailored to detect deepfakes for a specific speaker. For our GP classifier, we used the learned kernel with 20 samples—10 real and 10 fake—from a single individual to create a personalized detector. For the baseline methods, we fine-tuned them on the same 20 samples. To adapt to a new, unseen TTS, we used samples generated from that TTS for our personalized speaker, following the same approach as in Sec. 4.2 with MixPro. We show our results in Table. 2 for test speakers from LibriFake dataset, as well as new speakers taken from VoxCeleb [31]. Each experiment was repeated 30 times on 30 different speakers.

As one can see, in general, all personalized methods generalize better to the unseen 11Labs TTS samples when compared to their general detector counterpart. For our ADD-GP method, we achieve 0.61% EER with a 1-shot adaptation on LibriFake, comparable to a 100-shot adaptation in the non-personalized experiment. It is important to note that while this personalized detector exhibits excellent performance, it is limited by the fact that it requires deepfake samples specifically for the target speaker rather than relying on generic deepfake samples.

4.4. Calibration

An additional benefit of the Bayesian GP approach is that it offers an accurate measure of uncertainty, thereby providing users with a reliable measure of confidence. This is particu-

larly advantageous for ADD, as it informs users of the model’s confidence rather than receiving a simple binary decision. One way to measure uncertainty estimation is via calibration, where we see how the returned probabilities relate to accuracy. The predictions are binned according to $\max_y p(y|\mathbf{x})$, and each bin’s average probability is compared to the average accuracy of these samples. As shown in Fig. 1, the ADD-GP returns well-calibrated results. It is important to note that the SSL-ASSIST and CD-ADD baselines return a score and not a probability and, as such, cannot be evaluated using the same calibration metrics.

Algorithm 1 GP Deepfake Detection

Require: Dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, divide to $\mathcal{D}_{train}, \mathcal{D}_{eval}$, where $\mathbf{x}_i$ are speech samples and y_i are labels. A GP classifier with RBF kernel, Pretrained XLS-R model (only last block trainable).

- 1: **Training Phase:**
- 2: **for** $t = 1$ to T **do**
- 3: Sample batch of 80 examples from $\mathcal{D}$
- 4: Compute embeddings $\mathbf{z} = f_{\text{XLS-R}, \theta}(\mathbf{x})$ using XLS-R
- 5: Fit a GP classifier using RBF kernel:

$$k(\mathbf{z}_i, \mathbf{z}_j) = \sigma^2 \exp \left(-\frac{\|\mathbf{z}_i - \mathbf{z}_j\|^2}{2\ell^2} \right)$$

- 6: Compute log-marginal likelihood $\mathcal{L}$
- 7: Compute gradients $\nabla \mathcal{L}$ and update trainable XLS-R parameters θ , the length scale ℓ and the output scale σ
- 8: **end for**
- 9: **Evaluation Phase:**
- 10: Load the 1000 reserved training examples $\mathcal{D}_{eval}$, and compute embeddings $\mathbf{z}_{eval} = f_{\text{XLS-R}, \theta^*}(\mathbf{x}_{eval})$
- 11: Predict using $p(\mathbf{f}_* | \mathbf{z}_*, \mathbf{y}_{eval}, \mathbf{z}_{eval})$
- 12: Compute classification metrics on the test set

5. Conclusion

In this work, we address the issue of adapting audio deepfake detection to new unseen TTSS. As demonstrated by our experiments, this can be a significant security concern, as ADD performance on seen TTS is almost perfect while degrading considerably on unseen TTSS. We tackle this issue by introducing a novel ADD approach based on Gaussian processes, which we name ADD-GP. Our experiments show that this model can adapt better to new TTSS with relatively few samples. It also has additional benefits, such as better personalization and returning well-calibrated predictions. This shows how this can be an important tool to address the dangers of deepfake-based fraud in a dynamic and evolving environment.

6. References

- [1] C. Wang, S. Chen, Y. Wu, Z. Zhang, L. Zhou, S. Liu, Z. Chen, Y. Liu, H. Wang, J. Li *et al.*, “Neural codec language models are zero-shot text to speech synthesizers,” *arXiv preprint arXiv:2301.02111*, 2023.
- [2] M. Le, A. Vyas, B. Shi, B. Karrer, L. Sari, R. Moritz, M. Williamson, V. Manohar, Y. Adi, J. Mahadeokar *et al.*, “Voicebox: Text-guided multilingual universal speech generation at scale,” *Advances in neural information processing systems*, vol. 36, 2024.
- [3] E. Casanova, J. Weber, C. D. Shulby, A. C. Junior, E. Gölge, and M. A. Ponti, “Youtrts: Towards zero-shot multi-speaker tts and zero-shot voice conversion for everyone,” in *International Conference on Machine Learning*. PMLR, 2022, pp. 2709–2720.
- [4] Y. Chen, Z. Niu, Z. Ma, K. Deng, C. Wang, J. Zhao, K. Yu, and X. Chen, “F5-tts: A fairytaler that fakes fluent and faithful speech with flow matching,” *arXiv preprint arXiv:2410.06885*, 2024.
- [5] O. A. Shaaban, R. Yildirim, and A. A. Alguttar, “Audio deepfake approaches,” *IEEE Access*, vol. 11, pp. 132 652–132 682, 2023.
- [6] E. J. Hu, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen *et al.*, “Lora: Low-rank adaptation of large language models,” in *International Conference on Learning Representations (ICLR)*, 2022.
- [7] A. Babu, C. Wang, A. Tjandra, K. Lakhotia, Q. Xu, N. Goyal, K. Singh, P. Von Platen, Y. Saraf, J. Pino *et al.*, “Xls-r: Self-supervised cross-lingual speech representation learning at scale,” *arXiv preprint arXiv:2111.09296*, 2021.
- [8] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*. The MIT Press, 2006.
- [9] J. Hensman, A. Matthews, and Z. Ghahramani, “Scalable variational gaussian process classification,” in *Artificial Intelligence and Statistics*. PMLR, 2015, pp. 351–360.
- [10] A. G. Wilson, Z. Hu, R. Salakhutdinov, and E. P. Xing, “Stochastic variational deep kernel learning,” in *Proceedings of the 30th International Conference on Neural Information Processing Systems*, 2016, pp. 2594–2602.
- [11] I. Achituve, G. Chechik, and E. Fetaya, “Guided deep kernel learning,” in *Uncertainty in Artificial Intelligence*. PMLR, 2023, pp. 11–21.
- [12] N. G. Polson, J. G. Scott, and J. Windle, “Bayesian inference for logistic models using Pólya–Gamma latent variables,” *Journal of the American Statistical Association*, pp. 1339–1349, 2013.
- [13] T. Galy-Fajou, F. Wenzel, C. Donner, and M. Opper, “Multi-class Gaussian process classification made conjugate: Efficient inference via data augmentation,” in *Uncertainty in Artificial Intelligence*. PMLR, 2020, pp. 755–765.
- [14] I. Achituve, A. Navon, Y. Yemini, G. Chechik, and E. Fetaya, “GP-Tree: A Gaussian process classifier for few-shot incremental learning,” in *International Conference on Machine Learning*. PMLR, 2021, pp. 54–65.
- [15] I. Achituve, A. Shamsian, A. Navon, G. Chechik, and E. Fetaya, “Personalized federated learning with Gaussian processes,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 8392–8406, 2021.
- [16] D. Milios, R. Camoriano, P. Michiardi, L. Rosasco, and M. Filippone, “Dirichlet-based Gaussian processes for large-scale calibrated classification,” *Advances in Neural Information Processing Systems*, vol. 31, 2018.
- [17] R. Calandra, J. Peters, C. E. Rasmussen, and M. P. Deisenroth, “Manifold Gaussian processes for regression,” in *2016 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2016, pp. 3338–3345.
- [18] A. G. Wilson, Z. Hu, R. Salakhutdinov, and E. P. Xing, “Deep kernel learning,” in *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2016.
- [19] J. Yi, C. Wang, J. Tao, X. Zhang, C. Y. Zhang, and Y. Zhao, “Audio deepfake detection: A survey,” *arXiv preprint arXiv:2308.14970*, 2023.
- [20] J.-w. Jung, H.-S. Heo, H. Tak, H.-j. Shim, J. S. Chung, B.-J. Lee, H.-J. Yu, and N. Evans, “Aasist: Audio anti-spoofing using integrated spectro-temporal graph attention networks,” in *arXiv preprint arXiv:2110.01200*, 2021.
- [21] —, “Aasist: Audio anti-spoofing using integrated spectro-temporal graph attention networks,” in *IEEE international conference on acoustics, speech and signal processing (ICASSP)*, 2022, pp. 6367–6371.
- [22] H. Tak, J. weon Jung, J. Patino, M. Kamble, M. Todisco, and N. Evans, “End-to-end spectro-temporal graph attention networks for speaker verification anti-spoofing and speech deepfake detection,” in *Proc. 2021 Edition of the Automatic Speaker Verification and Spoofing Countermeasures Challenge*, 2021, pp. 1–8.
- [23] J.-w. Jung, Y. J. Kim, H.-S. Heo, B.-J. Lee, Y. Kwon, and J. S. Chung, “Pushing the limits of raw waveform speaker recognition,” *Proc. Interspeech*, 2022.
- [24] S. Schneider, A. Baevski, R. Collobert, and M. Auli, “wav2vec: Unsupervised pre-training for speech recognition,” *arXiv preprint arXiv:1904.05862*, 2019.
- [25] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, “Hubert: Self-supervised speech representation learning by masked prediction of hidden units,” *IEEE/ACM transactions on audio, speech, and language processing*, vol. 29, pp. 3451–3460, 2021.
- [26] Y. Li, M. Zhang, M. Ren, M. Ma, D. Wei, and H. Yang, “Cross-domain audio deepfake detection: Dataset and analysis,” *arXiv preprint arXiv:2404.04904*, 2024.
- [27] X. Li, P.-Y. Chen, and W. Wei, “Sonar: A synthetic ai-audio detection framework and benchmark,” *arXiv preprint arXiv:2410.04324*, 2024.
- [28] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, “Librispeech: an asr corpus based on public domain audio books,” in *2015 IEEE international conference on acoustics, speech and signal processing (ICASSP)*. IEEE, 2015, pp. 5206–5210.
- [29] E. Kharitonov, D. Vincent, Z. Borsos, R. Marinier, S. Girgin, O. Pietquin, M. Sharifi, M. Tagliasacchi, and N. Zeghidour, “Speak, read and prompt: High-fidelity text-to-speech with minimal supervision,” 2023. [Online]. Available: <https://arxiv.org/abs/2302.03540>
- [30] Z. Zhang, L. Zhou, C. Wang, S. Chen, Y. Wu, S. Liu, Z. Chen, Y. Liu, H. Wang, J. Li *et al.*, “Speak foreign languages with your own voice: Cross-lingual neural codec language modeling,” *arXiv preprint arXiv:2303.03926*, 2023.
- [31] A. Nagrani, J. S. Chung, and A. Zisserman, “Voxceleb: a large-scale speaker identification dataset,” *arXiv preprint arXiv:1706.08612*, 2017.
- [32] Y. Xue, A. Payani, Y. Yang, and B. Mirzasoleiman, “Few-shot adaption to distribution shifts by mixing source and target embeddings,” *International Conference on Machine Learning (ICML)*, 2024.
- [33] H. Zhang, “mixup: Beyond empirical risk minimization,” *arXiv preprint arXiv:1710.09412*, 2017.
- [34] H. Tak, M. Todisco, X. Wang, J.-w. Jung, J. Yamagishi, and N. Evans, “Automatic speaker verification spoofing and deepfake detection using wav2vec 2.0 and data augmentation,” *arXiv preprint arXiv:2202.12233*, 2022.
- [35] G. Zeng, Y. Chen, B. Cui, and S. Yu, “Continual learning of context-dependent processing in neural networks,” *Nature Machine Intelligence*, vol. 1, no. 8, pp. 364–372, 2019.
- [36] X. Zhang, J. Yi, C. Wang, C. Y. Zhang, S. Zeng, and J. Tao, “What to remember: Self-adaptive continual learning for audio deepfake detection,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 17, 2024, pp. 19 569–19 577.